BURSA TEKNİK ÜNİVERSİTESİ

MÜHENDİSLİK VE DOĞA BİLİMLERİ FAKÜLTESİ

BİLGİSAYAR MÜHENDİSLİĞİ BÖLÜMÜ

Node.JS ile Web Programlama

LAB_4

22360859088
KORAY GARİP

1. Fonksiyon Syntaxları

Bu aşamada JavaScript'te fonksiyon tanımlama yöntemleri olan normal fonksiyon (function) ve arrow function (=>) yapıları incelendi. İki yapı arasındaki en önemli farkın this kullanımı olduğu gözlemlendi.

Normal fonksiyonlarda this değeri, fonksiyonun nasıl çağrıldığına bağlı olarak belirlenir. Eğer bir fonksiyon bir obje içinde tanımlanır ve o obje üzerinden çağrılırsa, this o objeyi temsil eder. Örneğin bir obje içinde tanımlanan normal bir fonksiyon, this anahtar kelimesi ile aynı obje içerisindeki verilere erişebilir.

Buna karşılık arrow function'lar kendi this değerine sahip değildir. Arrow function içinde kullanılan this, fonksiyonun bulunduğu parent scope'tan alınır. Bu nedenle bir obje içinde arrow function kullanıldığında, this o objeyi değil, daha dıştaki scope'u referans alır ve bu durumda tanımlı bir erişim yoksa undefined döndürmesine yol açabilir.

Bu yüzden özellikle obje içinde tanımlanan ve this kullanımı gerektiren fonksiyonlarda function kullanılması gerekmektedir. Arrow function'lar ise daha çok kısa ve pratik kullanım gereken, this kullanılmayan durumlarda tercih edilir.

Alıştırma 1

Arrow function'lar ile daha kısa ve pratik kullanılan örnek yapıldı.

```
const tasks={
    tasks :[{
        text:"Market Alışverişi",
        completed:true
    },
    {
        text:"Ev Temizliği",
        completed:false
    },
    {
        text:"Bulaşık Yıkama",
        completed:false
    }
],
    tamamlanmis(){
        return this.tasks.filter((task)=> task.completed === true)
    },
    tamamlanmamis(){
        return this.tasks.filter((task)=> task.completed === false)
    }

}
```

2. Notebook Uygulaması Geliştirme

Geliştirilen not uygulaması üzerine devam edilerek yargs ile komut yapısı genişletildi. Daha önce ekleme ve silme işlemleri yapılmıştı, bu hafta ise listeleme (list) fonksiyonu eklendi. Bu fonksiyon sayesinde notes.json dosyasında saklanan tüm notlar okunarak konsola yazdırıldı. Uygulamada önceden yazdığımız loadNotes fonksiyon kullanılarak veriler fs modülü ile JSON dosyasından okunmuş ve JSON.parse() ile JavaScript nesnesine dönüştürülmüştür. Elde edilen veri üzerinde forEach döngüsü ve arrow function kullanılarak tüm notlar tek tek gezilmiş ve listeleme işlemi gerçekleştirilmiştir. Eğer listelenecek veri bulunmuyorsa kullanıcıya bilgilendirme mesajı verilmiştir. Listeleme sırasında her bir notun başlık ve içerik bilgisi konsola yazdırılmıştır. Ayrıca çıktıların daha anlaşılır ve görsel olarak ayırt edilebilir olması için chalk kütüphanesi kullanılarak konsol mesajları renklendirilmiştir.

```
}
const listNotes=()=>{
    const notes=loadNotes()
    if(notes.length<1){ return console.log("Listelenecek veri yok") }
    notes.forEach(element => {console.log(chalk.magenta(`Baslik: ${element.title} - İcerik: ${element.body}`))
  });

}

module.exports={
    loadNotes:loadNotes,
    addNote: addNote,
    removeNotes:removeNote,
    listNotes:listNotes
}
```

```
PROBLEMS   OUTPUT   DEBUG CONSOLE   TERMINAL   PORTS        powershell - notes

PS C:\Users\koray\Desktop\nodeproject\notes> node .\app.js list
Baslik: node - İcerik: js
Baslik: rapor - İcerik: lab raporu teslimi
PS C:\Users\koray\Desktop\nodeproject\notes> node .\app.js list
Baslik: node - İcerik: js
Baslik: rapor - İcerik: lab raporu teslimi
PS C:\Users\koray\Desktop\nodeproject\notes>
Ln 51, Col 50   Spaces: 4   UTF-8   CRLF   { } JavaScript
```

```
yargs.command({
    command:"list",
    describe:"Notlar listelendi",
    handler : ()=>{
        notes.listNotes()
    }

})
```